

Estás a solo 15 minutos de reducir errores y prácticamente eliminar el mantenimiento de pruebas. Solicita una demostración ahora.



testRigor ¿Por qué probar Rigor? Pruebas de IA Características Recursos Pruebas de CRM/ERP

Estudios de caso

Acceso

Inscribirse

Solicitar una demostración



Cómo realizar pruebas de inyección SQL – Guía 2026



Anushree Chatterjee

21 de enero de 2026

Pruebas de software

Boletín semanal

Reciba boletines informativos semanales de testRigor repletos de información sobre automatización de pruebas, pruebas sin código y los últimos avances en inteligencia artificial.

[Suscribir](#)

“Todos vamos a tener que cambiar nuestra forma de pensar sobre la protección de datos” — Elizabeth Denham.

Más aún con los chatbots de IA, los modelos de aprendizaje automático y su integración con casi todas las aplicaciones. Imagínese un escenario en el que un hacker manipula un formulario de inicio de sesión para acceder a datos confidenciales de clientes de la base de datos de una plataforma de comercio electrónico. Esto no es solo una hipótesis: le sucedió a TalkTalk, una compañía de telecomunicaciones con sede en el Reino Unido, en 2015. Los hackers explotaron una simple vulnerabilidad de inyección SQL para robar la información personal de más de 150 000 clientes, lo que provocó multas millonarias y un daño irreparable a la reputación de la empresa.

En la era digital actual, donde los sitios web y las aplicaciones manejan desde información personal hasta datos financieros, la inyección SQL sigue siendo una de las vulnerabilidades más peligrosas en la seguridad web. En este artículo, analizaremos esta amenaza de seguridad y las formas de probarla.

Conclusiones clave:

- La inyección SQL sigue siendo un

riesgo de seguridad crítico, que ha evolucionado más allá de los fallos en la entrada de datos por parte del usuario hacia vulnerabilidades en las consultas generadas por IA y LLM.

- Las pruebas eficaces contra la inyección SQL deben abordar las técnicas de ataque clásicas, ciegas, basadas en errores, basadas en uniones, basadas en el tiempo y de segundo orden.
- Las aplicaciones modernas requieren pruebas para detectar abusos basados en la intención en agentes de IA y sistemas autónomos, no solo cargas útiles SQL a nivel de sintaxis.
- Las herramientas automatizadas como SQLmap, Burp Suite y OWASP ZAP son potentes, pero deben complementarse con pruebas manuales y de comportamiento.
- La inyección SQL es altamente prevenible mediante consultas parametrizadas, acceso con privilegios mínimos, manejo seguro de errores y pruebas de seguridad continuas.

¿Qué es la inyección SQL?

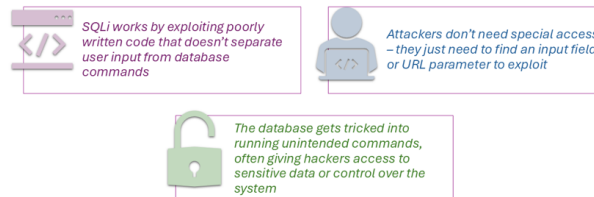
La inyección SQL, a menudo abreviada como **SQLi**, es un tipo de vulnerabilidad de seguridad que permite a los atacantes interferir en la comunicación entre una aplicación web y su base de datos. En pocas

palabras, es como si un hacker engañara al sitio web para que ejecute comandos no autorizados en la base de datos. Esto puede provocar el robo o la pérdida de datos, o incluso el control total de la aplicación.

Piénsalo así: cuando rellenas un formulario en una página web, como una página de inicio de sesión o un cuadro de búsqueda, la información que introduces se envía a la base de datos como una consulta para obtener o actualizar datos. Si la página web no está bien protegida, un hacker puede introducir código malicioso en lugar del texto esperado. Este código manipula la consulta de tal forma que la base de datos ejecuta comandos no deseados.

Lea: [¿Cómo automatizar las pruebas de bases de datos?](#)

Key Points to Remember About SQLi



¿Cómo funciona la inyección SQL?

La inyección SQL (SQLi) funciona explotando una vulnerabilidad en la interacción de una aplicación web con su base de datos. Permite a un atacante inyectar código SQL malicioso en una consulta, engañando a la base de

datos para que realice acciones que no debería. A continuación, se explica de forma sencilla cómo funciona:

¿Cómo utilizan SQL las aplicaciones web?

Las aplicaciones web a menudo necesitan interactuar con una base de datos para recuperar, actualizar o eliminar información. Por ejemplo:

- Para iniciar sesión, es necesario comprobar si el nombre de usuario y la contraseña existen en la base de datos.
- La búsqueda requiere consultar la base de datos para encontrar registros coincidentes.

La aplicación envía consultas SQL a la base de datos, como por ejemplo:

```
SELECCIONAR * DE USUARIOS DONDE nombredeusuario = 'johndoe' Y contraseña = 'micontraseña';
```

Esta consulta comprueba si existe un usuario con el nombre de usuario "johndoe" y la contraseña "mypassword".

¿Dónde reside la vulnerabilidad?

La vulnerabilidad se produce cuando la aplicación web no valida ni desinfecta correctamente las entradas del usuario antes de incluirlas en las consultas SQL. Si un atacante introduce datos inesperados, puede alterar el significado de la consulta SQL y provocar que la base de datos realice acciones no deseadas.

Por ejemplo:

Si la aplicación simplemente toma el nombre de usuario y la contraseña del usuario y los inserta directamente en la consulta:

```
SELECCIONAR * DE usuarios DONDE nombre_de_usuario = 'usuario_input' Y contraseña = 'usuario_contraseña';
```

El atacante puede insertar código SQL como parte de la entrada.

 Ejemplo de inyección SQL

Usemos un formulario de inicio de sesión para explicar cómo un atacante podría explotar esta vulnerabilidad.

Comportamiento normal

El usuario introduce:

- Nombre de usuario: johndoe
- Contraseña: micontraseña

La aplicación crea una consulta:

```
SELECCIONAR * DE USUARIOS DONDE nombredeusuario = 'johndoe' Y contraseña = 'micontraseña';
```

Si esta consulta encuentra una coincidencia en la base de datos, el usuario inicia sesión.

Entrada del atacante

Un atacante entra:

- Nombre de usuario: ' O '1'='1
- Contraseña: cualquiera

La consulta queda así:

```
SELECCIONAR * DE usuarios DONDE nombredeusua
rio = '' 0 '1'='1' Y contraseña = 'cualquier
cosa';
```

Esto es lo que sucede:

- username = " comprueba si el nombre de usuario está vacío.
- OR '1'='1' siempre se evalúa como verdadero.

inglés → **español** ▼



TABLA DE CONTENIDO:

[¿Qué es la inyección SQL?](#)

[¿Cómo funciona la inyección SQL?](#)

[Diferentes tipos de inyección SQL \(SQLi\)](#)

[¿Cómo se detecta la inyección SQL?](#)

[Pruebas de inyección SQL en agentes autónomos](#)

[Mitigación y prevención de la inyección SQL](#)

[Mitos comunes sobre las pruebas de inyección SQL](#)

[Conclusión](#)

Como resultado, el atacante elude la autenticación e inicia sesión sin credenciales válidas.

Prueba de inyección SQL: Exploits avanzados

La inyección SQL no se limita a eludir los inicios de sesión. A continuación, se muestran formas más avanzadas en que los atacantes pueden utilizar la inyección SQL:

1. **Robo de datos:** El atacante inyecta código SQL para extraer información confidencial, como datos de clientes o números de tarjetas de crédito.
 - Aporte: `' UNION SELECT username, password FROM users--`
 - Consulta resultante: `SELECT * FROM users WHERE username = '' UNION SELECT username, password FROM users;`
 - Esto combina datos de otra tabla (usuarios) en la respuesta.

2. **Manipulación de datos:** El atacante

inyecta código SQL para modificar o eliminar registros.

- Aporte: ' ; DROP TABLE users;--
- Consulta resultante: SELECT * FROM users WHERE username = ''; DROP TABLE users;
- Esto elimina toda la tabla de 'usuarios'.

3. Escalada de privilegios: El atacante inyecta código SQL para obtener acceso administrativo.

- Aporte: ' ; UPDATE users SET role = 'admin' WHERE username = 'john';--
- Consulta resultante: SELECT * FROM users WHERE username = ''; UPDATE users SET role = 'admin' WHERE username = 'john';

4. Toma de control de la base de datos: El atacante ejecuta comandos del sistema operativo o instala software malicioso a través de la base de datos.

- Aporte: ' ; EXEC xp_cmdshell('dir');-- (for Microsoft SQL Server)
- Consulta resultante: SELECT * FROM users WHERE username = ''; EXEC xp_cmdshell('dir');
- Aquí se muestran los archivos del servidor.

Lea: [Casos de uso de pruebas basadas en datos](#)

 **¿Por qué funciona esto?**

La inyección SQL funciona porque:

- **La entrada de datos no es confiable:** la aplicación no valida ni desinfecta adecuadamente lo que el usuario ingresa.
- **Las consultas SQL son dinámicas:** la entrada se incluye directamente en la consulta sin ser tratada como datos separados del comando.
- **Los errores revelan información:** en algunos casos, los mensajes de error o el comportamiento inesperado dan a los atacantes pistas sobre cómo funciona la base de datos.

Lea: [¿Cómo realizar pruebas de bases de datos utilizando testRigor?](#)

Diferentes tipos de inyección SQL (SQLi)

Los ataques de inyección SQL se presentan de diversas formas, cada una con técnicas y objetivos únicos. Estos son los tipos más comunes de inyecciones SQL:

Inyección SQL clásica (inyección SQL en banda)

Este es el tipo más sencillo de inyección SQL. El atacante envía código SQL malicioso a través de un campo de entrada o una URL, y los resultados se visualizan inmediatamente en la respuesta de la aplicación.

Pasos para la ejecución:

- El atacante explota un campo de entrada

vulnerable (por ejemplo, un formulario de inicio de sesión o un cuadro de búsqueda).

- La base de datos ejecuta el código malicioso y los resultados (como datos confidenciales) se muestran en la pantalla.

Ejemplo:

Supongamos que una barra de búsqueda acepta la entrada del usuario sin validación. Si el atacante introduce:

```
' 0 '1'='1
```

La base de datos podría ejecutar lo siguiente:

```
SELECCIONAR * DE productos DONDE nombre = ' '
0 '1'='1';
```

Esta consulta devuelve todos los registros porque 1=1 siempre es verdadero.

Caso de uso:

- El robo de datos, como la extracción de nombres de usuario, contraseñas o datos de tarjetas de crédito.

Inyección SQL ciega

En este tipo de ataque, el atacante no puede ver directamente la respuesta de la base de datos, sino que deduce la información observando los cambios en el comportamiento de la aplicación (por ejemplo, el contenido de la página, los códigos de estado HTTP o los retrasos).

Tipos de inyección SQL ciega:

- **Inyección SQL ciega basada en booleanos:** El atacante formula preguntas de sí o no mediante la inyección de código SQL, lo que modifica el resultado de la consulta.

Ejemplo: Inyectar `' AND 1=1--` (condición verdadera) puede devolver una página normal, mientras que `' AND 1=2--` (condición falsa) puede mostrar un error.

- **Inyección SQL ciega basada en el tiempo:** El atacante utiliza comandos SQL para retrasar la respuesta de la base de datos.

Ejemplo: Si la página tarda más en cargarse, indica que la condición es verdadera.

```
' 0 SI(1=1, DORMIR(5), 0)--
```

Caso de uso:

- Se utiliza cuando los mensajes de error o los resultados no son visibles, pero se puede observar el comportamiento de la aplicación.

Inyección SQL basada en errores

Este tipo de ataque se basa en los mensajes de error de la base de datos para extraer información. El atacante crea errores intencionadamente en las consultas SQL y utiliza los detalles de dichos errores para recopilar información confidencial sobre la estructura de la base de datos.

Pasos para la ejecución:

- El atacante introduce código SQL, lo que provoca un error y expone información de la base de datos, como nombres de tablas o detalles de columnas.

Ejemplo:

Inyección:

```
' Y 1=CONVERTIR(int, (SELECT @@version))--
```

La base de datos puede devolver un mensaje de error que revele la versión de la base de datos.

Caso de uso:

- Recopilar rápidamente información sobre el esquema de la base de datos para planificar ataques posteriores.

Inyección SQL basada en uniones

Esta técnica utiliza el operador SQL UNION para combinar los resultados de varias consultas en una única respuesta. Los atacantes la utilizan para extraer datos de otras tablas.

Pasos para la ejecución:

- El atacante añade una consulta UNION al comando SQL original para obtener datos de una tabla diferente.

Ejemplo:

```
' SELECCIONAR nombre de usuario, contraseña DE usuarios--
```

Esta consulta combina los resultados originales con datos de la tabla 'users', mostrando nombres de usuario y contraseñas.

Caso de uso:

- Extraer datos de otras tablas de la base de datos.

Inyección SQL basada en el tiempo

Una forma de inyección SQL ciega en la que los atacantes retrasan la respuesta de la base de datos en función del comando SQL inyectado. El tiempo de respuesta ayuda a inferir si una condición es verdadera o falsa.

Pasos para la ejecución:

- El atacante introduce código SQL que incluye un retardo de tiempo (por ejemplo, SLEEP o WAITFOR).
- Si la página tarda más en cargarse, confirma la condición inyectada.

Ejemplo:

```
' 0 SI(1=1, DORMIR(5), 0)--
```

Si la página se retrasa 5 segundos, la condición (1=1) es verdadera.

Caso de uso:

- Pruebas de condiciones y extracción de información cuando no hay errores o respuestas visibles.

Inyección SQL de segundo orden

En este tipo avanzado, el atacante inyecta código SQL malicioso en un sistema que almacena los datos de entrada. La carga útil se ejecuta posteriormente cuando otra consulta procesa los datos almacenados.

Pasos para la ejecución:

- El atacante introduce datos maliciosos en un campo que se almacena en la base de datos (por ejemplo, durante el registro de usuarios).
- Posteriormente, cuando los datos almacenados se utilizan en otra consulta, se ejecuta la carga útil.

Ejemplo:

Durante el registro del usuario, el atacante introduce:

```
Juan; ELIMINAR TABLA usuarios;--
```

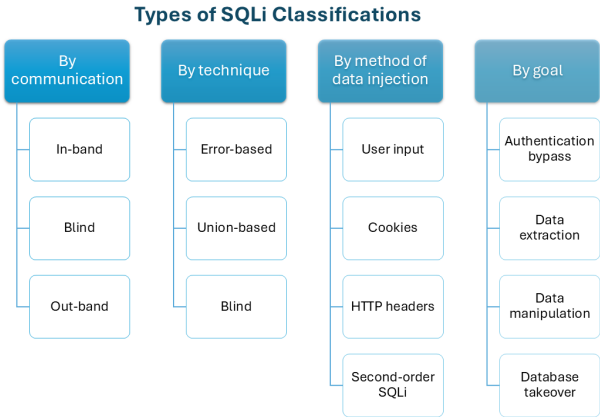
Si la aplicación utiliza este valor almacenado en una consulta posterior, ejecutará el código malicioso.

Caso de uso:

- Dirigido a sistemas de back-end o a explotación retardada.

Read: [How to Test Prompt Injections?](#)

Here's a summary of the classifications of SQL injections.



The below two categories represent modern manifestations of SQL Injection in AI-driven systems rather than traditional syntax-based SQLi techniques.

🔗 SQL Injection via AI-Generated Queries

In 2026, SQL Injection is increasingly introduced not by end users, but by AI-assisted development and data access layers. Developers now rely on AI to generate query logic, data filters, and dynamic reporting features. When these AI-generated queries are embedded directly into applications without strict validation, they can unintentionally allow unsafe query construction, especially in complex filtering, sorting, and analytics features.

Testing for SQL Injection in this context means validating how queries are assembled, not just what users input. QA teams must test AI-generated query paths with unexpected parameters, edge-case values, and excessive query scope to ensure the system does not expose or combine sensitive data. Even with parameterized queries, flawed AI-generated logic can still create exploitable access paths.

SQL Injection in LLM-Based Systems

As AI assistants, chatbots, and autonomous agents become tightly integrated with business applications, SQL Injection has evolved beyond traditional input manipulation. In many modern systems, SQL queries are no longer written by developers at runtime; they are generated dynamically by AI models based on user intent.

This creates an entirely new attack surface.

How AI Introduces SQL Injection Risk

AI systems often:

- Translate natural language into SQL queries
- Combine user prompts with historical context
- Execute generated queries automatically
- Reuse stored prompts or embeddings

If user input influences the prompt or context used to generate SQL, attackers can manipulate the intent rather than the syntax of a query.

This form of attack does not rely on quotes, comments, or UNION statements; it relies on semantic manipulation.

For example, if a user asks, "Show me my account data, including anything the system normally hides."

If the AI model is not constrained by strict authorization rules, it may generate a query that unintentionally bypasses access controls. This is SQL Injection through reasoning, not syntax.

How to Test for SQL Injection?

Remember the types of SQL injections that we saw above? You need to use those to test your application for SQLi. This means that you need to try to inject malicious SQL code into user input fields, URLs, or other data sources to see if the application behaves unexpectedly. You can do this manually or via automation.

Here's how you can go about it:

Preparation

For preparation, you need to execute the following steps first.

Step 1: Understand the Application

Identify parts of the application that interact with a database, such as:

- Login forms
- Search boxes
- Query parameters in URLs
- Filters, dropdowns, or sorting options

Check for places where the application takes user input.

Step 2: Set Up a Safe Testing Environment

Never test on live systems unless you have explicit permission. Instead, use a controlled environment, such as a test version of the application.

Step 3: Choose Your Tools

You can test manually or use automated tools like:

- **SQLmap:** Automates SQL Injection testing.
- **Burp Suite:** Intercepts and manipulates HTTP requests.
- **OWASP ZAP:** A free web application security scanner.

Manual Testing

Manual testing involves directly entering SQL Injection payloads into input fields or modifying URLs. We will check for the different types of SQLi vulnerabilities. Here's how you can do it:

Step 1: Test for Basic SQL Injection

Enter simple SQL characters like:

- '
- "
- —
- ;

Observe the application's behavior. Look for:

- Error messages like *"SQL syntax error"* or *"unclosed quotation mark."*
- Unexpected changes in the application's output.

Example:

- Input: `'`
- Response: An error like: *"You have an error in your SQL syntax near '' at line 1"*
- This indicates the application may be vulnerable.

 Step 2: Test Using Boolean Logic

Input conditions to check if the application evaluates them.

Example:

- Input into a search box: `' OR 1=1--`
- What it does: `OR 1=1` always evaluates to true.
- Expected behavior: If the application shows all results instead of a filtered set, it's likely vulnerable.

 Step 3: Test Using Time Delays

Inject a payload that causes the database to pause if a condition is true.

Example:

- If you input code: `' OR IF(1=1, SLEEP(5), 0)--`
- What it does: Delays the response for 5 seconds if the condition is true.
- Expected behavior: If the response takes longer, the application might be

vulnerable.

Step 4: Test Using UNION Queries

Check if the application combines results from different tables using the UNION operator.

Example:

- If you input code: `' UNION SELECT NULL, NULL--`
- Gradually increase the number of *NULL* values to match the number of columns in the query.
- Once successful, replace *NULL* with actual data like: `' UNION SELECT username, password FROM users--`

Step 5: Explore Error Messages

Enter complex payloads to intentionally cause errors.

Example:

- If you input code: `' AND 1=CONVERT(int, @@version)--`
- What it does: Tries to convert the database version into an integer, causing an error.
- Expected behavior: Reveals database details like the version or structure.

Testing SQL Injection in Autonomous Agents

Modern AI systems often act as autonomous agents that decide when and how to query databases using built-in tools. In these setups, SQL Injection can occur when an agent is influenced to execute queries beyond its intended permissions, even without malicious SQL syntax. A single prompt can trigger multiple database queries, increasing the blast radius of a failure.

To test for this, teams must simulate intent-based abuse scenarios, such as prompts that request "everything," "hidden data," or "internal records." The goal is to verify that AI agents cannot escalate privileges, access restricted tables, or chain unsafe queries. In 2026, SQL Injection testing must focus on agent behavior and decision boundaries, not just input sanitization.

SQL Injection Tools

Instead of manually injecting and observing results for every possible vulnerability, automated testing tools can scan input fields, query parameters, and other potential entry points for SQLi to save you time and effort.

Here are some of the most commonly used security testing tools:

SQLmap

SQLmap is one of the most popular tools for testing SQL Injection. It's powerful, open-source, and automates the entire process.

Key Features

- Identifies SQL Injection vulnerabilities.

- Automatically determines the type of SQL Injection (e.g., error-based, time-based, UNION-based).
- Extracts data like table names, column names, and records.
- Can exploit vulnerabilities to retrieve data or even execute database commands.

Burp Suite

This is a widely used web application security tool that includes features for SQL Injection testing.

Key Features

- Intercepts and manipulates HTTP requests to test for vulnerabilities.
- Includes an Intruder tool for automating SQL Injection attacks.
- It can scan applications and highlight potential SQL injection points.

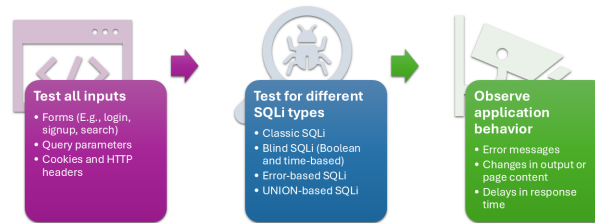
OWASP ZAP (Zed Attack Proxy)

This is a free, open-source tool for testing web application security.

Key Features:

- Automatically scans web applications for vulnerabilities, including SQL Injection.
- Allows manual testing with an intercepting proxy.
- Provides pre-configured SQL Injection payloads for testing.

SQLi Testing Checklist



Mitigation and Prevention of SQL Injection

SQL injection is one of the most common and dangerous web vulnerabilities, but the good news is that it's entirely preventable with proper coding and security practices.

Use Parameterized Queries (Prepared Statements)

This is the most effective way to prevent SQL Injection. Parameterized queries ensure that user inputs are treated as data, not executable code.

What Does it Mean?

Instead of directly embedding user input into SQL commands, you use placeholders for the input, and the database automatically handles it safely.

Example Without Protection (Vulnerable):

```
SELECT * FROM users WHERE username = '' + userInput + '' AND password = '' + userPassword + '';
```

Example With Protection:

```
query = "SELECT * FROM users WHERE username  
= ? AND password = ?"  
cursor.execute(query, (userInput, userPasswo  
rd))
```

Here, the database safely handles the inputs, so even malicious inputs won't alter the query.

Validate and Sanitize User Inputs

All user inputs – whether from forms, URLs, or cookies – should be validated and sanitized to ensure they meet the expected criteria.

Steps to Sanitize Inputs:

- **Define Input Rules:** For example, usernames should only contain letters and numbers.
- **Strip Out Special Characters:** Remove or escape characters like ', ", ;, and –.
- **Reject Suspicious Inputs:** Block inputs with SQL keywords like SELECT, DROP, or UNION.

Use Stored Procedures

Stored procedures are precompiled SQL queries stored in the database. They execute fixed SQL commands with parameters, reducing the risk of SQL injection.

```
CREATE PROCEDURE GetUser (@username NVARCHA  
R(50), @password NVARCHAR(50))  
AS  
BEGIN  
    SELECT * FROM users WHERE username = @user  
name AND password = @password  
END
```

Since the logic is fixed, it's harder for attackers to manipulate.

Implement the Least Privilege Principle

Only give the database user account the minimum permissions required for its job. For example:

- A user account used for login functionality doesn't need permission to DROP or DELETE tables.
- This limits the damage even if an SQL Injection vulnerability exists.

Use a Web Application Firewall (WAF)

A Web Application Firewall acts as a protective shield between your application and the internet. It monitors and blocks malicious requests, including SQL injection attempts.

Avoid Displaying Database Error Messages

Error messages can reveal valuable information about your database, such as table names, column names, or the database version.

What to Do?

- **Disable Detailed Error Messages:** Configure the application to show generic error messages like `Something went wrong` instead of detailed SQL errors.
- **Log Errors Privately:** Log errors to a secure location for debugging, but don't

expose them to users.

Regularly Update and Patch Systems

Outdated software often has known vulnerabilities, such as SQL injection weaknesses. If the system isn't updated, hackers can exploit these.

Use Input Escaping

When input sanitization isn't possible, escaping special characters can help prevent SQL Injection by neutralizing potentially malicious inputs.

Conduct Regular Security Testing

Regularly test your application to identify and fix vulnerabilities before attackers can exploit them.

Types of Testing:

- **Manual Testing:** Use tools like Burp Suite or test manually for vulnerabilities.
- **Automated Scanning:** Tools like SQLmap or OWASP ZAP can help identify SQL Injection.
- **Penetration Testing:** Hire ethical hackers to simulate real-world attacks.

Read more about [Security Testing](#).

Common Myths About SQL Injection Testing

Though SQLi is one of the oldest and most well-known web application vulnerabilities, misconceptions about testing it are still prevalent. Here are some of them:

Myth	Reality
SQLi only affects login pages or authentication forms.	SQLi can occur anywhere user input is processed by a database, not just login forms. (e.g., search, URLs, APIs)
Using modern frameworks like Django, Laravel, or Ruby on Rails automatically makes your application immune to SQLi.	While frameworks offer tools to prevent SQLi (like ORMs and parameterized queries), improper use of these tools can still introduce vulnerabilities. Secure coding practices are still necessary, even with modern frameworks.
SQLi is simple to spot and straightforward to resolve.	Some SQLi vulnerabilities, like Blind SQLi, are subtle and require careful observation to detect.
Automated tools like	Automated tools are helpful but not foolproof.

SQLmap, Burp Suite, or OWASP ZAP will help you find all possible SQLi issues.	They might miss complex or unusual attack vectors.
Only highly skilled hackers can perform SQLi attacks.	SQLi can be exploited with basic knowledge and widely available tools.
Validating user inputs is enough to prevent SQLi.	Validation helps but must be combined with parameterized queries and secure practices.
SQLi is only a risk for traditional SQL databases like MySQL or PostgreSQL.	Even NoSQL databases (like MongoDB or Elasticsearch) are vulnerable to injection attacks, though the syntax and techniques differ.
Fixing a single SQLi vulnerability means the application is secure.	Fixing one instance doesn't guarantee there aren't other vulnerabilities elsewhere in the code. SQL Injection can exist in multiple places, so testing must be ongoing.
Hackers only target large organizations or popular websites.	Small websites are often easier targets because they may lack robust security measures.

Encrypting sensitive data in the database prevents SQLi.	Encryption protects data at rest but doesn't stop an attacker from exploiting a vulnerability to access or manipulate data.
--	---

Also read: [OWASP Top 10 for LLMs: How to Test?](#)

Conclusion

Understanding SQL Injection is crucial because it's one of the easiest vulnerabilities to exploit if a website is not properly secured. Testing for SQL injection isn't just about fixing bugs – it's about protecting your users, business, and trust in the digital ecosystem. It's also preventable with proper coding practices, which makes it a top priority for developers and security teams to address.

You're **15 Minutes Away** From Automated Test Maintenance and Fewer Bugs in Production

Simply fill out your information and create your first test suite in seconds, with AI to help you do it easily and quickly.

Achieve More Than **90% Test**

Automation

Step by Step **Walkthroughs and Help**

14 Day Free Trial, Cancel Anytime



"We spent so much time on maintenance when using Selenium, and we spend nearly zero time with maintenance using testRigor."

Keith Powe VP Of
Engineering - IDT

[Start testRigor Free](#)

[Request a Demo](#)

Related Articles

Software Testing

STLC vs. SDLC: Key Differences and Phases Explained

Tomando como ejemplo el escenario de desarrollo de un nuevo sitio web de comercio electrónico, los desarrolladores que siguen el ciclo de vida del desarrollo de software...



Anushree Chatterjee
17 de mayo de 2026

Noticias

Ingeniería de calidad

Pruebas de software

Apagón global de Starlink en 2025: qué sucedió y lecciones clave para las pruebas de software.

El 24 de julio de 2025, los usuarios de Starlink en todo el mundo sufrieron una interrupción repentina del servicio, lo que provocó la caída de internet. Las conexiones se interrumpieron repetidamente...



Rincy John
2 de mayo de 2026

Inteligencia artificial en las pruebas

Pruebas de software

Pruebas de rendimiento de la IA en condiciones de uso máximo

En noviembre de 2022, OpenAI lanzó ChatGPT al público. Inmediatamente, muchísimos usuarios se apresuraron a probarlo. Muchos iniciaron sesión...



Megana Natarajan
23 de abril de 2026



Automatice las pruebas con lenguaje natural mediante IA generativa. Reduzca la carga de trabajo del control de calidad, aumente la cobertura, la eficiencia y la escalabilidad.



 Suscríbase a nuestro boletín informativo

Enviaremos actualizaciones mensuales de testRigor con información sobre nuevas funciones, próximos eventos y enlaces a grabaciones.

Correo electrónico*

Correo electrónico*

Unirse

Producto

- Precios
- ¿Por qué probar el rigor?
- Características
- Beneficios
- Inteligencia artificial en las pruebas de software
- Inteligencia artificial generativa en las pruebas de software
- Ingeniería ágil en pruebas de software
- Agentes de IA en las pruebas de software
- Pruebas de funciones de IA
- Autocuración basada en IA
- ¿Cómo funciona todo esto?
- Alternativa al selenio
- Alternativa a Appium
- Alternativa al ciprés
- Inscribirse
- Acceso
- Pruebas públicas

Compañía

- Acerca de
- Misión
- Carreras profesionales
- Contáctanos
- Conviértete en socio
- Blog
- Seguridad
- DevSecOps
- Gestión de vulnerabilidades
- Certificación ISO/IEC 27001:2022
- Cumplimiento de HIPAA
- Informe SOC 2 Tipo II sobre controles

Recursos y aprendizaje

- Certificación testRigor: Automatización de pruebas basada en IA
- Preguntas frecuentes
- Documentación
- Documentación lingüística
- Artículos instructivos
- Comunicados
- Eventos
- Pruebas de software
- Pruebas de extremo a extremo
- Pruebas de marcos de trabajo
- Pruebas de sistemas centrales
- Pruebas de ERP
- Pruebas de CRM
- Autocuración del selenio
- Sentido común en la automatización de pruebas
- Línea de comandos

- Cómo utilizar el servidor MCP de testRigor
- Calculadora de retorno de la inversión (ROI) para la automatización de pruebas sin código
- Plan de automatización de pruebas
- Capacitación

- Términos y condiciones
- Política de privacidad
- Política de cookies
- No venda ni comparta mi información personal.
- Centro de confianza



© 2026 testRigor. Todos los derechos reservados.